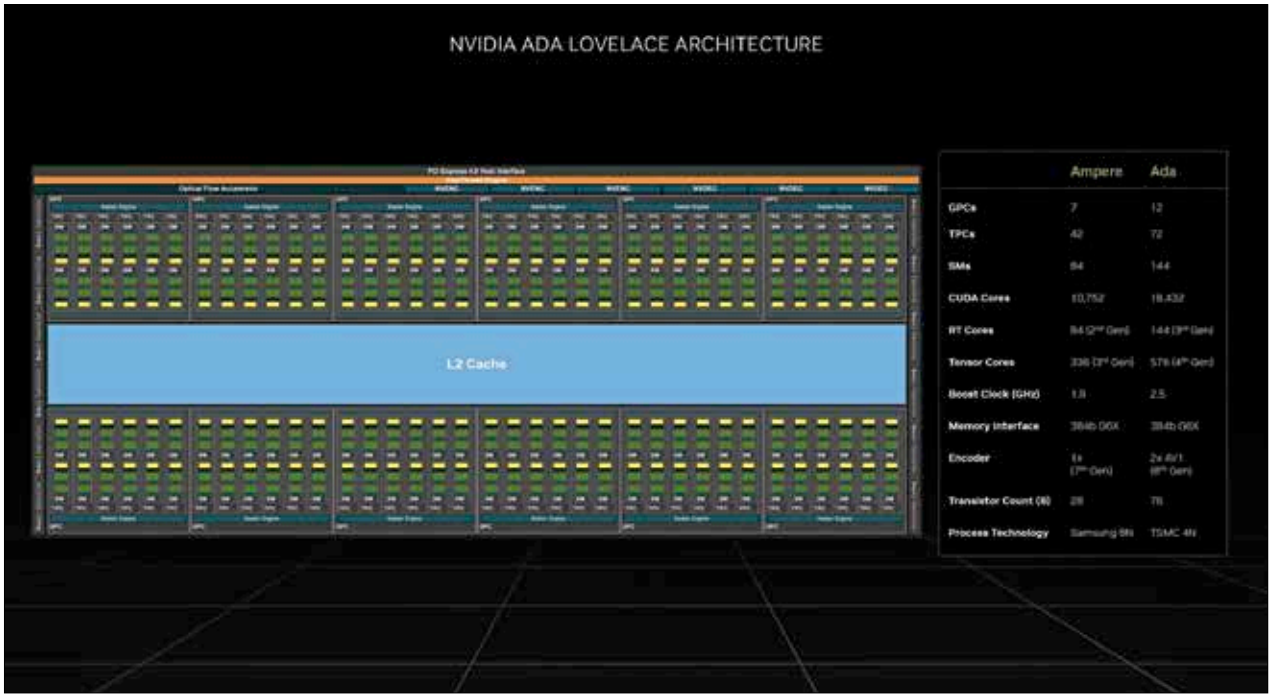
A hand of brotherhood
Google widgets for iOS 16 lock screen are available now for iPhones.
https://dgit.in/oct22-54

New SIM for 5G in India?
The short answer is — yes your 4G SIM is enough to get 5G service, once the latter is available. https://dgit.in/oct22-56



CUDA cores in the new Ada Lovelace architecture

graphics cards are more optimised for parallelised workloads. So you need to decide which type of workloads are assigned to the host and which need to remain with the devices.

There will be points in the code where the parallelisation has to pause because there is a dependency on some other workload or different threads need to synchronise with each other to share data. If these threads are from the same block then using _syncthreads() allows the threads to share data through shared memory within the same kernel invocation. If the threads are from different blocks then you have to share data using global memory which is a little slower and takes more kernel invocations.

Device Level

Each NVIDIA graphics card has several multiprocessors and each multiprocessor has several CUDA cores within them. Whenever you are writing an application, it should be written to maximise parallel execution between the multiprocessors of the device. The benefit here is that the multiprocessors are much closer to each other and share lower level cache memories. This allows for the workload to be executed much quicker without the need to move across devices.

Multiprocessor Level

Similarly, at the multiprocessor level, there are various functional units which all should be utilised to the maximum before moving to a higher level. A warp is the smallest unit of execution on an NVIDIA device and each warp is processed across 32 threads. Utilisation is closely linked to the number of resident warps and the warp scheduler selects an instruction to execute whenever it is ready. If the instruction to be executed is waiting for another instruction within the same warp then the two can be executed together within the same warp. We've already touched upon Asynchronous

MAXIMISE MEMORY THROUGHPUT

The way you maximise memory throughput is by minimising all the data transfers which are low bandwidth. The host and the device have a much slower interconnect than most of the other interconnects that we are concerned with. Hence, minimising the data transfers between the host and the device is what we need to do. This also entails minimising data transfers between global memory and the device memory by utilising more of the on-chip memory, the caches and the shared memory.

Shared memory is something that the application explicitly allocates and accesses. Typically, you would want to load data from global memory to device memory. This is followed by loading data from device memory to shared memory. Then you would want to synchronise with all the threads of the block so that the threads can read the shared memory locations that have been loaded by different threads. Then the processing of said data can begin. After that the shared memory needs to be updated with the results post processing. This requires another synchronisation. Finally, the results will be written back on the device memory.

In the case of certain applications, a traditional hardware-managed cache is more appropriate rather than having to manually optimise the